Somos F5

Introducción al Web Scraping

Extracción· estructura · paginación · guardado y ética

ABRIMOS RUTAS INCLUSIVAS AL TALENTO DIGITAL

Hoja de ruta

Somos F5

01

¿Qué es el Web Scraping?

Peticiones HTTP, HTML, BeautifulSoup y casos de uso

02

Anatomía de una petición HTTP

Headers, status codes, respuesta y errores comunes

03

HTML y selectores CSS

Etiquetas, clases, ids y selectores avanzados

04

Estructuras repetidas

find() vs find_all() y navegación por el árbol HTML

05

Paginación y guardado de datos

Bucle multipágina, CSV y JSON con pandas

06

Manejo de errores

try/except, reintentos y logging

07

Comparativa de herramientas

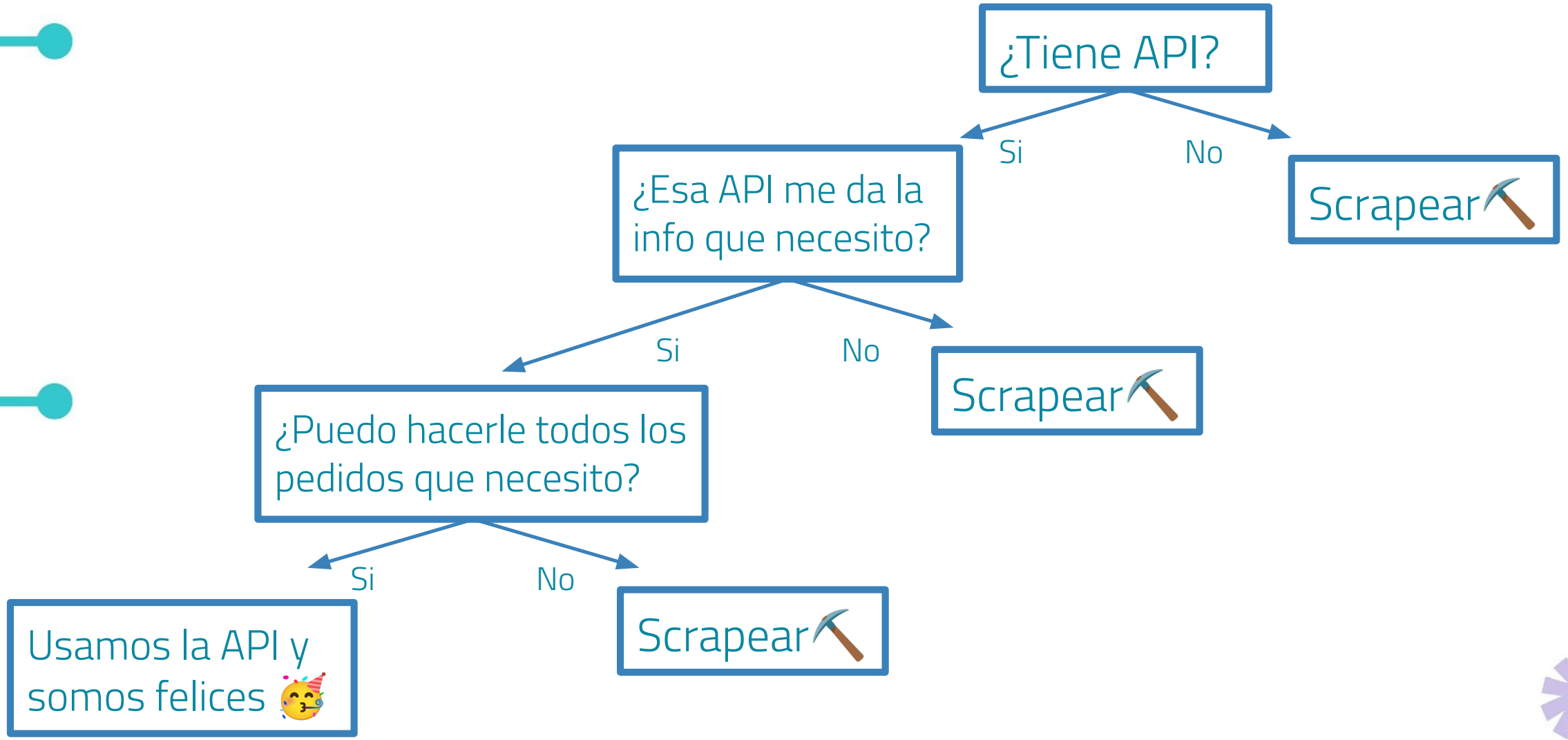
BS4 vs Scrapy vs Selenium — cuándo usar cada una

08

Ética y consideraciones técnicas

robots.txt, rate limiting, legalidad y buenas prácticas

API vs Web scraping (II)

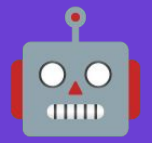


¿Qué es el Web Scraping

Somos F5

Tú, a mano

Abres la web de
El Corte Inglés, buscas
el precio del iPhone y
lo copias en Excel.
30 min de trabajo.



Web Scraping

Le dices a Python:
'Ve a esa web, busca
todos los precios
y guárdalos.'
3 segundos de trabajo.

Resultado

Tienes 1000 precios
en un DataFrame.
Analizas tendencias,
competencia, alertas
de precio.

Pipeline del Web Scraping

Tu script Python → Petición HTTP GET → Servidor web → Respuesta HTML → BS4 parsea → Datos extraídos

Casos de uso reales

- Comparadores de vuelos
- Portales inmobiliarios
- Agregadores de noticias
- Bolsas de trabajo
- Monitoreo de precios

Herramientas: requests (peticiones HTTP) + BeautifulSoup (parseo HTML) + pandas (guardado)

Anatomía de una petición HTTP

Somos F5

📤 REQUEST (lo que envías)

Método:	GET / POST / PUT / DELETE
URL:	https://books.toscrape.com/page-1
Headers:	User-Agent, Accept, Cookies...
Body:	(solo en POST — datos del formulario)

⇄

📥 RESPONSE (lo que recibes)

Status:	200 OK / 403 / 404 / 429
Headers:	Content-Type, Set-Cookie...
Body:	<html>...</html> (el HTML)

Códigos de estado HTTP más importantes

200	301	403	404	429	500
OK	**Redirección**	**Prohibido**	**No encontrado**	**Too Many Requests**	**Error servidor**
Todo bien — extrae datos	requests la sigue sola	La web bloquea bots	La URL no existe	Añade time.sleep()	Reintenta más tarde

HTML: El idioma de las páginas web

```html
<!DOCTYPE html>
<html>
  <body>
    <h1 class="titulo">
      Laptop Dell XPS
    </h1>
    <p class="precio">
      999.99€
    </p>
    <a href="/producto/1">
      Ver detalles
```

¿Qué busca nuestro script?

🏷️ `<h1 class="titulo">`

Etiqueta de título. El atributo class nos permite encontrarlo con BS4.

🔑 `<p class="precio">`

Párrafo con clase "precio". Buscamos soup.find("p", class_="precio")

🔗 `<a href="/producto/1">`

Enlace. El atributo href tiene la URL destino.

📝 `class / id`

Son selectores: como coordenadas GPS del dato en el HTML.

💡 *F12 en tu navegador → Inspeccionar → ¡Estás viendo el HTML en vivo!*

Estructuras Repetidas: el patrón es tu amigo

```
# Patrón que se repite en el HTML:
<div class="producto">
  <h2>Laptop Dell XPS</h2>
  <span class="precio">999€</span>
  <a href="/producto/1">Ver más</a>
</div>
<div class="producto">
  <h2>MacBook Air</h2>
  <span class="precio">1299€</span>
  <a href="/producto/2">Ver más</a>
</div>
```

Paginación y guardado

🔗 Patrón de URL: https://ejemplo.com/productos?pagina= 1 → 2 → 3 → ...

```python
import time

todos_datos = []

for pagina in range(1, 51):  # 50 págs
  url = f"...page-{pagina}.html"
  r = requests.get(url)
  soup = BeautifulSoup(r.text)
```

```python
    datos = {
      "titulo": item.h3.text,
      "precio": item.find(
        "p",class_="price").text
    }
    todos_datos.append(datos)
  time.sleep(1)  # ¡respeta!
```

💾 Al terminar el bucle → df = pd.DataFrame(todos_datos) → df.to_csv("datos.csv", index=False)

Guardar los datos: CSV vs JSON

Somos F5

📋 CSV

- ✅ Compatible con Excel y pandas
- ✅ Fácil de visualizar
- ✅ Ligero y universal
- ⚠️ Solo datos tabulares planos
- ⚠️ No soporta anidamiento

```
df = pd.DataFrame(datos)
df.to_csv(
  "productos.csv",
  index=False,
  encoding="utf-8-sig"
```

VS

📦 JSON

- ✅ Soporta estructuras anidadas
- ✅ Nativo en APIs y web
- ✅ Flexible y autodescriptivo
- ⚠️ Más pesado que CSV
- ⚠️ Menos directo para pandas

```
import json
with open("datos.json","w") as f:
  json.dump(
    datos, f,
    ensure_ascii=False
```

Manejo de errores: scrapers que no se rompen

Somos F5

Timeout	403 Forbidden	429 Too Many	HTML cambiado	Error de red
timeout=10 en requests.get()	Añadir User-Agent headers	time.sleep() + reintentos	try/except AttributeError	ConnectionError + reintentos

```python
import logging, time, requests
logging.basicConfig(level=logging.INFO)


def get_con_reintentos(url, reintentos=3, pausa=1):
  for i in range(1, reintentos + 1):
    try:
      time.sleep(pausa)
      return requests.get(url, timeout=10)
```

Nuestras herramientas

Somos F5

requests

El camarero

Va a la URL que le indiques y te trae el HTML de la página. Como hacer una llamada a una web.

BeautifulSoup

El chef

Toma ese HTML crudo y lo convierte en algo fácil de navegar. Filtra y extrae exactamente lo que necesitas.

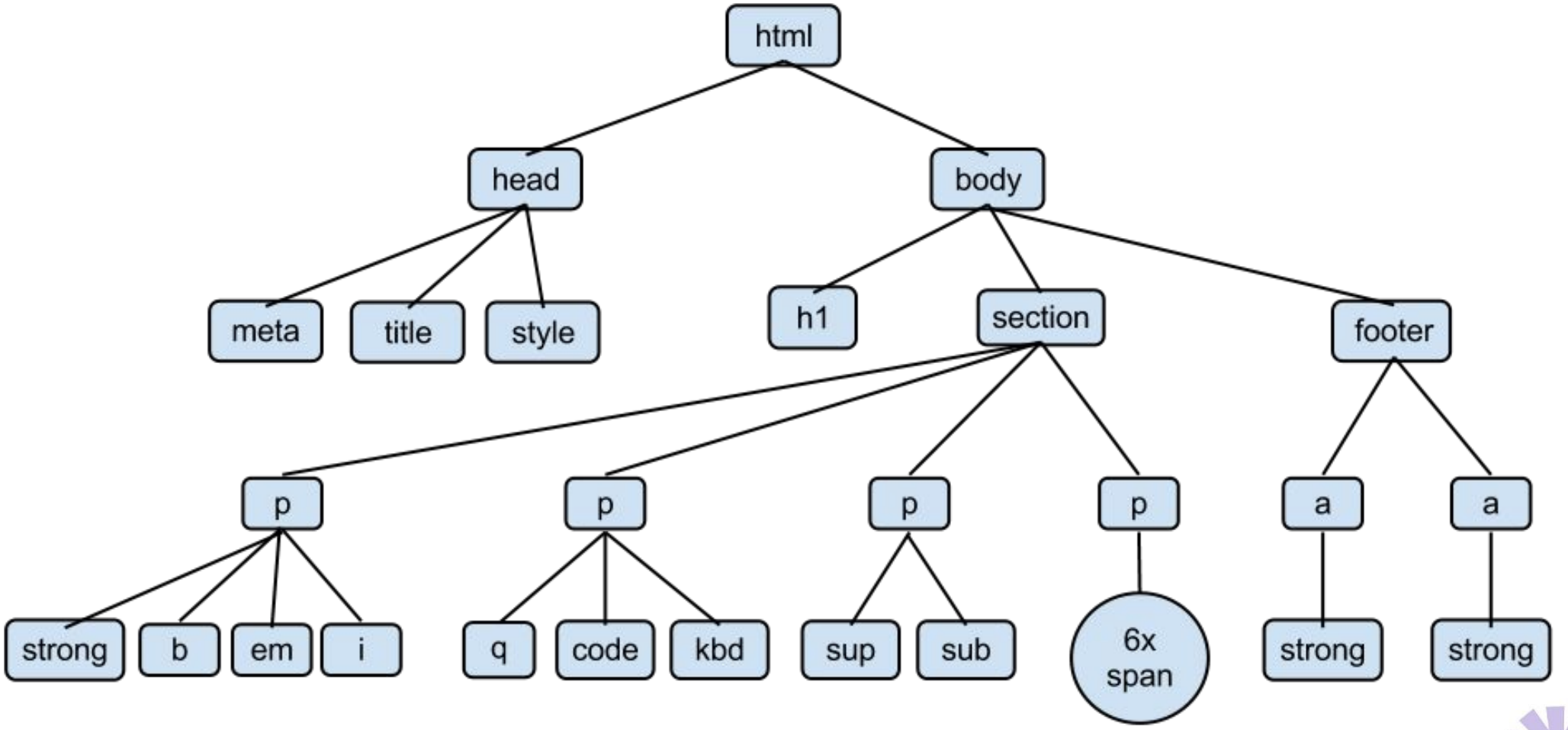
pandas

La nevera

Guarda los datos extraídos en un DataFrame. Perfecto para analizar, filtrar y exportar a CSV o Excel.

```
$ pip install requests beautifulsoup4 pandas
```

Parsear con Beautiful Soup



Somos F5

🐍 requests + BS4

✅ Fácil de aprender

✅ Sin configuración

✅ Perfecto para empezar

❌ No ejecuta JS

❌ Sin concurrencia nativa

¿Cuándo?

Webs estáticas simples
Pocos cientos de páginas

⭐ **Principiante**

🌐 Selenium / Playwright

✅ Ejecuta JavaScript

✅ Interacción real

✅ Webs dinámicas

❌ Lento

❌ Necesita navegador

¿Cuándo?

SPAs (React, Vue)
Formularios y clics

⭐⭐ **Intermedio**

🕷 Scrapy

✅ Muy rápido (async)

✅ Pipelines y middlewares

✅ Escala bien

❌ Curva de aprendizaje alta

❌ No ejecuta JS nativo

¿Cuándo?

Miles de páginas
Proyectos de producción

⭐⭐⭐ **Avanzado**

¿Qué pasa con webs dinámicas?

Somos F5

⚠️ Muchas webs modernas usan JavaScript para cargar datos DESPUÉS de que el HTML llega. requests solo ve el HTML inicial, no los datos dinámicos.

✅ requests + BeautifulSoup

Webs estáticas

→ HTML cargado en el servidor

→ Tiendas simples, blogs, noticias

→ Rápido y ligero

→ Lo que hemos aprendido hoy

🚀 Selenium / Playwright

Webs dinámicas

→ JavaScript renderiza en el cliente

→ Instagram, LinkedIn, Google Maps

→ Controla un navegador real

→ Más lento, más potente

Ética y legalidad: las reglas del Web Scraping

Somos F5

robots.txt

Indica qué rutas el propietario permite o prohíbe rastrear.

Comprueba siempre:
https://sitio.com/robots.txt

Rate Limiting

No sobrecargas el servidor. Enviar peticiones muy rápido puede ser ilegal (DoS).

Regla de oro:
time.sleep(1) entre peticiones

Legalidad

Datos públicos sin login: OK con moderación. Personales o tras login: ilegal.

RGPD aplica.
Siempre lee los Terms of Service.

¿Hay API?

Si existe API oficial, úsala siempre. Es más estable, legal y eficiente.

Twitter, Reddit, GitHub,
Spotify → API oficial.

✅ Buenas prácticas vs ❌ Lo que debes evitar

BUENAS PRÁCTICAS		LO QUE DEBES EVITAR
✅ Leer y respetar robots.txt		❌ Ignorar robots.txt
✅ Añadir time.sleep() entre peticiones		❌ Bucle sin pausa (sobrecarga el servidor)
✅ Usar headers con User-Agent real		❌ Scrapear datos personales sin base legal
✅ Scrapear solo datos públicos	VS	❌ Extraer contenido detrás de login
✅ Usar API oficial si existe		❌ Saltarse CAPTCHAs con automatización
✅ Manejar errores con try/except		❌ Ignorar el código 429
✅ Guardar datos de forma incremental		❌ Revender datos scrapeados
✅ Cachear para no repetir peticiones		❌ Ignorar los Terms of Service

"El scraping responsable es técnicamente posible y legalmente viable. Solo requiere sentido común."

Web Scraping + IA : El combo perfecto

Somos F5

Análisis de sentimiento

Scraping de reseñas de Amazon ⟶ Entrenar modelo NLP para clasificar opiniones positivas/negativas

Predicción de precios

Scraping diario de Idealista ⟶ Dataset para modelo de predicción de precios inmobiliarios

Búsqueda semántica

Scraping de artículos de noticias ⟶ Base de conocimiento para RAG con LLMs (como ChatGPT)

Entrenamiento de LLMs

Así se crearon los datasets de GPT: scraping masivo de Common Crawl, Wikipedia, libros...

Business Intelligence

Monitorización automática de competidores, precios, tendencias de mercado en tiempo real

Datos para investigación

Recolección de datos clínicos públicos, papers científicos, ensayos para modelos médicos

Lo que nos llevamos hoy

Somos F5

1 **HTTP + requests**

get() → status_code → headers → .text (HTML)

2 **BeautifulSoup**

find() / find_all() / select() → .text → ['atributo']

3 **Estructuras repetidas**

Patrón HTML + find_all() + bucle for

4 **Paginación + guardado**

range() + f-string + time.sleep() → CSV / JSON

5 **Manejo de errores**

try/except + reintentos + logging

6 **Herramientas**

BS4 → Scrapy → Selenium según la complejidad

7 **Ética**

robots.txt · rate limiting · ToS · API si existe

Somos F5

ABRIMOS RUTAS INCLUSIVAS
AL TALENTO DIGITAL